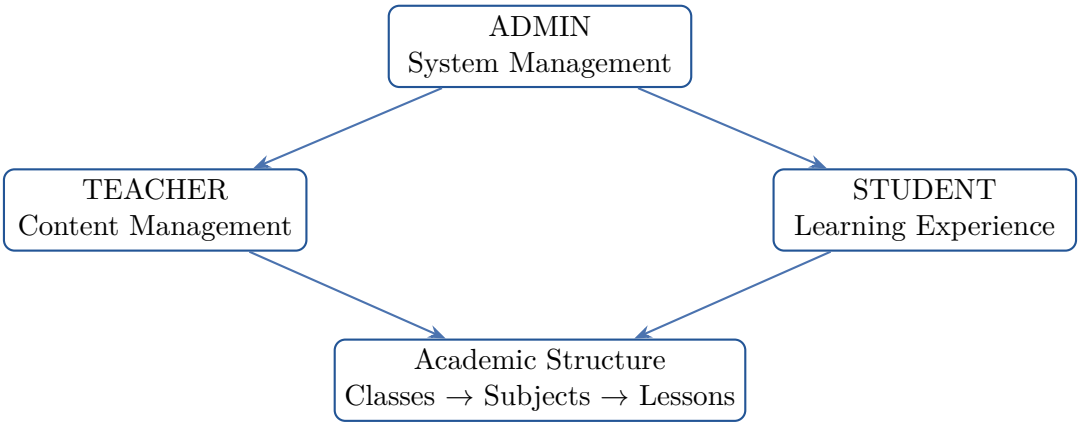
# AcademyAlys

Educational Learning Management Platform

Complete System Architecture

Project Handover and Technical Specification




**Core Principle**

Admin defines the academic structure and permissions.
Teachers create learning content within their assignments.
Students consume only the content authorized for their enrollment.

**Project Scope: Classes 6–10**

Version 1.0

2026

# Contents

**1 Project Overview** **4**

**2 User Roles** **4**
2.1 Admin . . . 4
2.2 Teacher . . . 5
2.3 Student . . . 5

**3 Complete System Architecture** **6**

**4 Recommended Technology Stack** **6**

**5 Academic Structure** **6**

**6 Database Architecture** **7**
6.1 Core Tables . . . 7
6.2 Student Data Model . . . 8
6.3 Teacher Data Model . . . 8
6.4 Subject Data Model . . . 9

**7 Learning Material Architecture** **9**

**8 Admin Portal** **10**
8.1 Admin Navigation . . . 10
8.2 Admin Dashboard . . . 10

**9 Admin Class Management** **11**
9.1 Class Selection . . . 11
9.2 Selected Class Tabs . . . 12
9.3 Class Summary . . . 12

**10 Student Management** **12**
10.1 Global Student Directory . . . 12
10.2 Class-Specific Student Management . . . 13

**11 Add Student Workflow** **13**
11.1 Student Fields . . . 14

**12 Bulk Student Import** **14**

**13 Teacher Management** **16**

**14 Teacher Creation and Assignment** **16**

**15 Teacher Access Control** **17**

**16 Teacher Portal** **17**

**17 Teacher Material Workflow** **18**
17.1 Video Upload . . . 19

**18 Student Portal** **19**

**19 Student Subject and Lesson Flow** **19**

**20 Student Lesson Learning Interface** **20**

**21 Student Video Experience** **20**

**22 Other Learning Materials** **21**
22.1 PDF . . . 21
22.2 PPT . . . 21
22.3 Notes . . . 21

**23 Authentication and Routing** **21**

**24 Authorization Architecture** **22**
24.1 Admin . . . 22
24.2 Teacher . . . 22
24.3 Student . . . 22

**25 API Architecture** **23**

**26 Frontend Architecture** **24**

**27 Backend Architecture** **25**

**28 Complete Project Structure** **25**

**29 File Storage** **27**

**30 Security Requirements** **27**

**31 Responsive Design** **28**
31.1 Desktop . . . 28
31.2 Tablet . . . 28
31.3 Mobile . . . 28

**32 UI/UX Principles** **28**

**33 Important Data Rules** **29**
33.1 Historical Enrollment . . . 29

**34 Future AI Learning Layer** **29**

**35 Development Roadmap** **30**
35.1 Phase 1 – Foundation . . . 30
35.2 Phase 2 – Academic Structure . . . 31
35.3 Phase 3 – User Management . . . 31
35.4 Phase 4 – Teacher Portal . . . 31
35.5 Phase 5 – Student Portal . . . 31
35.6 Phase 6 – Administration . . . 32
35.7 Phase 7 – AI . . . 32

**36 Complete End-to-End System Flow** **32**
36.1 Admin Flow . . . 32
36.2 Teacher Flow . . . 33
36.3 Student Flow . . . 33

**37 Permission Flow** **34**

**38 Final Acceptance Criteria** **34**

**39 Master Development Principle** **35**

**40 Master Handover Prompt** **36**

# 1 Project Overview

AcademyAlys is a role-based educational learning management platform designed for students in Classes 6–10.

The system provides three completely separate portals:

1. Admin Portal
2. Teacher Portal
3. Student Portal

The platform is designed around a controlled academic structure where administrators define the classes, subjects, lessons, users and permissions. Teachers then create learning content within their assigned classes and subjects, while students consume content according to their academic enrollment.

**Core System Principle**

$$\boxed{\text{ADMIN} \rightarrow \text{PERMISSIONS} \rightarrow \text{TEACHER} \rightarrow \text{CONTENT} \rightarrow \text{STUDENT}}$$

The application should function as a standard LMS even if the future AI layer is disabled.

# 2 User Roles

## 2.1 Admin

The Admin has system-wide administrative access.

Admin responsibilities include:

- Manage students.
- Manage teachers.
- Manage academic years.
- Manage classes.
- Manage sections.
- Manage subjects.
- Manage lessons.
- Assign students to classes and sections.
- Assign teachers to classes and subjects.
- Manage learning materials.
- Manage announcements.
- Manage timetable.
- View reports.
- View activity logs.
- Manage platform settings.

## 2.2 Teacher

Teachers do not have system-wide access.

A teacher can access only the class–subject combinations explicitly assigned by the Admin.

For example:

```
Teacher A

Class 10 -> English
Class 9  -> English
```

The teacher can access:

```
Class 10 -> English
Class 9  -> English
```

The teacher cannot access:

```
Class 10 -> Mathematics
Class 10 -> Science
Class 8  -> English
```

unless the Admin explicitly creates those assignments.

## 2.3 Student

Students are restricted according to their academic enrollment.

For example:

```
Student
    |
    +-- Class 10
          |
          +-- English
          +-- Mathematics
          +-- Science
          +-- Social Science
          +-- Tamil
```

The actual available subjects must be determined from the database rather than hardcoded into the frontend.

## 3 Complete System Architecture

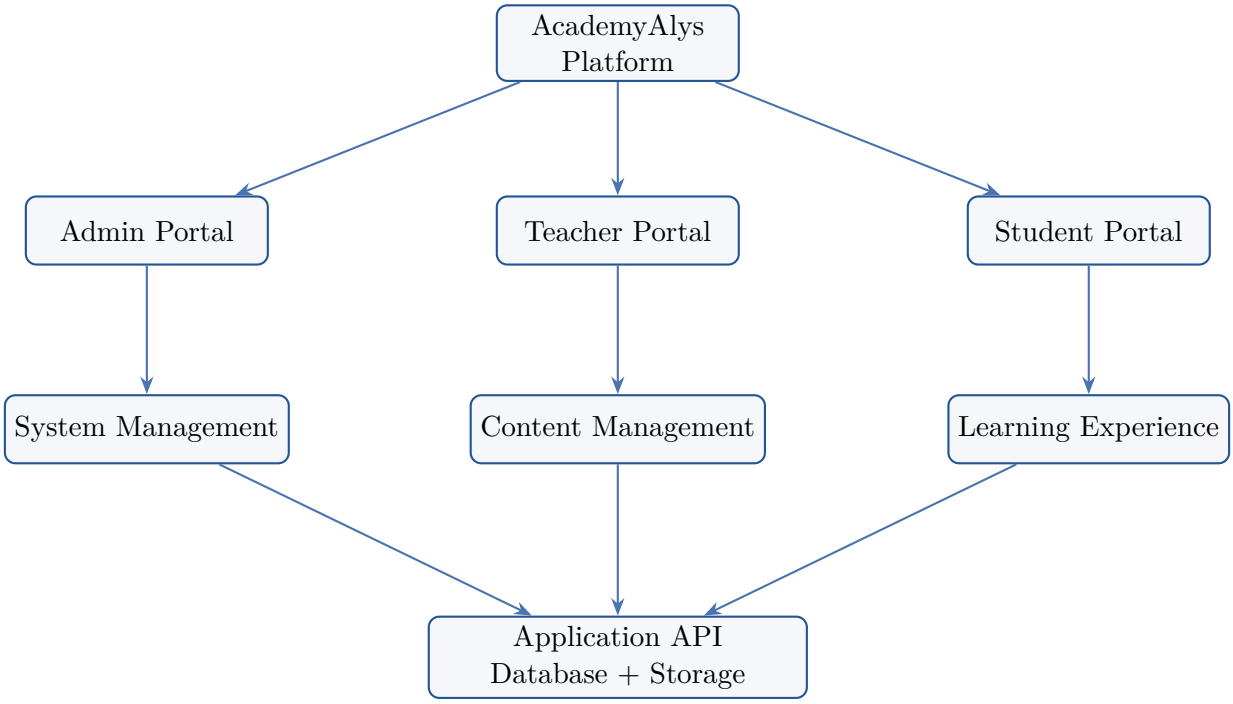



Figure 1: High-level AcademyAlys architecture

## 4 Recommended Technology Stack

| **Layer** | Technology |
|---|---|
| **Frontend** | React, Vite, TypeScript |
| **Styling** | Tailwind CSS |
| **Routing** | React Router |
| **Server State** | TanStack Query |
| **Backend** | FastAPI, Python |
| **Validation** | Pydantic |
| **ORM** | SQLAlchemy |
| **Database** | PostgreSQL |
| **Authentication** | Supabase Auth or equivalent secure authentication provider |
| **Storage** | Supabase Storage or S3-compatible object storage |
| **Frontend Deployment** | Vercel or equivalent |
| **Backend Deployment** | Render or equivalent |
| **Version Control** | Git and GitHub |

Table 1: Recommended technology stack

## 5 Academic Structure

The academic structure is:

```
Academic Year
      |
      v
    Class
```

```
      |
      v
   Section
      |
      v
   Subject
      |
      v
 Unit (optional)
      |
      v
   Lesson
      |
      v
 Topic (optional)
      |
      v
 Learning Material
```

For example:

```
Academic Year: 2026-2027
Class: 10
Subject: English
Unit: Unit 1 - Prose
Lesson: A Letter to God

Materials:
    - Video
    - Video
    - PDF
    - PDF
    - PPT
    - Notes
    - Practice Questions
```

# 6 Database Architecture

The application should use PostgreSQL as the primary relational database.

## 6.1 Core Tables

- users
- roles
- students
- teachers
- academic_years
- classes
- sections
- subjects

- class_subjects
- student_enrollments
- teacher_assignments
- units
- lessons
- topics
- materials
- announcements
- timetable
- activity_logs

## 6.2 Student Data Model

A student should be represented through separate account, profile and enrollment information.

```
User Account
     |
     v
Student Profile
     |
     v
Student Enrollment
     |
     +-- Academic Year
     +-- Class
     +-- Section
```

This allows academic history to be preserved.
For example:

```
2025-2026 -> Class 9
2026-2027 -> Class 10
```

Changing a student's class must not destroy previous enrollment records.

## 6.3 Teacher Data Model

```
User Account
     |
     v
Teacher Profile
     |
     v
Teacher Assignment
     |
     +-- Class
     +-- Subject
     +-- Optional Section
```

### 6.4 Subject Data Model

Subjects should not be assumed to be available to every class.

Use a class-subject mapping:

```
Subject
   |
   v
Class Subject
   |
   +-- Class 6  -> English
   +-- Class 6  -> Mathematics
   +-- Class 10 -> English
   +-- Class 10 -> Mathematics
```

## 7 Learning Material Architecture

Teachers can upload:

- Videos
- PDFs
- PPT presentations
- Notes
- Documents
- External links
- Other learning resources

Each material must contain or reference:

- Class
- Subject
- Lesson
- Teacher
- Material type
- Title
- Description
- Storage location or URL
- Publication status
- Created timestamp
- Updated timestamp

Example:

```
Material
    |
    +-- Class: 10
    +-- Subject: English
    +-- Lesson: A Letter to God
    +-- Teacher: Ms. Priya
    +-- Type: Video
    +-- Title: Complete Explanation
    +-- Status: Published
```

Large files such as videos and PDFs should not be stored directly in PostgreSQL. Use object storage.

# 8 Admin Portal

## 8.1 Admin Navigation

The Admin sidebar should contain:

```
Dashboard

User Management
    - Students
    - Teachers

Academic Structure
    - Class Management
    - Curriculum Subjects
    - Lesson Modules

Materials
    - All Materials
    - Review / Publishing

Timetable

Announcements

Reports & Analytics

Platform Settings

Academic Year Selector

Logout
```

## 8.2 Admin Dashboard

The dashboard should provide:

- Total Students.
- Total Teachers.
- Total Classes.

- Total Subjects.
- Quick actions.
- Recent users.
- Academic structure overview.
- Recent activity.
- Learning materials overview.
- Announcements.

Quick actions:

- Add Student
- Add Teacher
- Manage Classes
- Manage Subjects
- Manage Materials
- Create Announcement

All dashboard statistics must eventually come from the database.

# 9 Admin Class Management

Class Management is the central academic configuration workspace.

The Admin can manage Classes 6–10.

## 9.1 Class Selection

The workflow is:

```
Academic Structure
       |
       v
Class Management
       |
       v
Select Academic Year / Term
       |
       v
Select Class
       |
       +-- Class 6
       +-- Class 7
       +-- Class 8
       +-- Class 9
       +-- Class 10
```

Once a class is selected, it becomes the active management context.

### 9.2 Selected Class Tabs

```
Class 10

Overview
Students
Sections
Teachers
Subjects
Lessons
```

The selected class should remain active when navigating between these areas.

### 9.3 Class Summary

The class management page can display:

- Total enrolled students.
- Number of sections.
- Number of subjects.
- Number of teacher assignments.
- Class status.
- Academic year.

## 10 Student Management

There are two complementary student-management workflows.

### 10.1 Global Student Directory

Navigation:

```
User Management
       |
       v
Students
       |
       v
Student Directory
```

The global directory displays students across all classes.
Features:

- Search.
- Class filter.
- Section filter.
- Account status filter.
- Enrollment status filter.
- View profile.

- Edit student.
- Add student.
- Import students.
- Export students.
- Activate account.
- Deactivate account.

### 10.2 Class-Specific Student Management

Navigation:

```
Class Management
       |
       v
Select Class
       |
       v
Students
```

The selected class is automatically inherited.
The Admin should not be required to select the class again.

## 11 Add Student Workflow

If the Admin starts from the global Student Directory:

```
Add Student
     |
     v
Select Class
     |
     v
Basic Information
     |
     v
Section Assignment
     |
     v
Account Setup
     |
     v
Review
     |
     v
Create Student
```

If the Admin starts from Class Management:

```
Class 10
   |
   v
Students
```

```
   |
   v
Add Student
   |
   v
Basic Information
   |
   v
Section
   |
   v
Account Setup
   |
   v
Review
   |
   v
Create
```

The class must already be selected.

### 11.1 Student Fields

Possible fields include:

- Full Name
- Email
- Roll Number
- Date of Birth
- Contact Number
- Parent/Guardian Name
- Parent/Guardian Contact
- Student Photo
- Section
- Account Status

Passwords must never be displayed in plaintext.

## 12 Bulk Student Import

The bulk import workflow is:

```
Import Students
       |
       v
Select Class
       |
       v
Download Template
```

```
       |
       v
Upload Excel / CSV
       |
       v
Parse File
       |
       v
Validate
       |
       v
Detect Duplicates
       |
       v
Preview
       |
       v
Confirm
       |
       v
Import
```

Required fields:

- Full Name
- Email
- Roll Number
- Section

Optional fields:

- Contact Number
- Date of Birth
- Parent/Guardian Name

Validation should detect:

- Missing required fields.
- Invalid email formats.
- Duplicate emails.
- Duplicate roll numbers.
- Invalid sections.
- Invalid file structure.
- Unsupported file formats.

Invalid rows must not be silently skipped.

## 13 Teacher Management

Navigation:

```
User Management
       |
       v
Teachers
       |
       v
Teacher Directory
```

The Teacher Directory should show:

- Teacher Name
- Email
- Assigned Classes
- Assigned Subjects
- Account Status
- Date Added
- Actions

## 14 Teacher Creation and Assignment

Teacher creation follows:

```
Add Teacher
     |
     v
Basic Teacher Information
     |
     v
Select Class
     |
     v
Select Subject
     |
     v
Optional Section
     |
     v
Account Setup
     |
     v
Review
     |
     v
Create Teacher
```

A teacher can have multiple assignments.
Example:

```
Teacher A

Class 10 -> English
Class 9  -> English
```

Each assignment is an explicit database relationship.

## 15 Teacher Access Control

This is one of the most important security requirements.

The permission relationship is:

```
Teacher
   |
   v
Teacher Assignment
   |
   +-- Class
   |
   +-- Subject
   |
   +-- Optional Section
```

A teacher must not be able to access another class or subject simply by modifying a frontend URL or API request.

For example:

```
Teacher A
    |
    +-- Class 10
    |      |
    |      +-- English
    |
    +-- Class 9
           |
           +-- English
```

Teacher A cannot access:

```
Class 10 -> Mathematics
Class 10 -> Science
Class 8  -> English
```

unless those assignments exist.

## 16 Teacher Portal

After successful authentication:

```
Teacher Login
      |
      v
Teacher Dashboard
```

Teacher sidebar:

```
Dashboard

My Subjects

Upload Materials

My Materials

Timetable

Announcements

Profile

Logout
```

The dashboard must display only assigned classes and subjects.
Example:

```
My Subjects

Class 10
    English

Class 9
    English
```

## 17 Teacher Material Workflow

The teacher selects:

```
Class
  |
  v
Subject
  |
  v
Lesson
  |
  v
Material Type
```

Supported material types:

- Video
- PDF
- PPT
- Notes
- Document
- Link
- Other

### 17.1 Video Upload

```
Upload Video
     |
     v
Video Title
     |
     v
Description
     |
     v
Upload File OR Video Link
     |
     v
Preview
     |
     v
Publish
```

The backend must verify that the teacher has permission to upload content for the selected class and subject.

## 18 Student Portal

After login:

```
Student Login
      |
      v
Student Dashboard
      |
      v
Assigned Class
```

Example:

```
Student
   |
   +-- Class 10
```

The student's available subjects are retrieved from the class-subject configuration.

## 19 Student Subject and Lesson Flow

The student flow is:

```
Student Dashboard
       |
       v
Assigned Class
       |
       v
Subjects
       |
       v
```

```
English
       |
       v
Lessons
       |
       v
Lesson
```

Example:

```
English
   |
   +-- Unit 1: Prose
          |
          +-- Lesson 1
          +-- Lesson 2
          +-- Lesson 3
```

## 20 Student Lesson Learning Interface

When a student opens a lesson, display a two-column learning interface.

```
+-----------------------------------------------------+
|                  Lesson Header                      |
+----------------------+------------------------------+
| Lesson Sidebar       | Main Content                 |
|                      |                              |
| Home / Summary       | Selected Content             |
| Videos               |                               |
| PDF Materials        | Video Player / PDF Viewer    |
| PPT Presentations    |                               |
| Notes                |                               |
| Practice Questions   |                               |
+----------------------+------------------------------+
```

Example sidebar:

```
Lesson 1: A Letter to God

Home / Summary        8
Videos                2
PDF Materials         3
PPT Presentations     1
Notes                 2
Practice Questions    2
```

The numbers must be calculated dynamically from the database.

## 21 Student Video Experience

When the student clicks “Videos”:

```
Videos (2)

+-----------------------------------------+
```

```
|                                          |
|              VIDEO PLAYER                |
|                                          |
|        Teacher Uploaded Video            |
|                                          |
|    Play / Pause / Progress / Volume      |
|                                          |
+------------------------------------------+

Complete Explanation
Description
Uploaded by Teacher

-------------------------------------------

Animated Story
Thumbnail
Duration
Description
```

The API must return only videos satisfying:

$$\text{Student Class} + \text{Accessible Subject} + \text{Selected Lesson} + \text{Published Status}$$

# 22 Other Learning Materials

## 22.1 PDF

Provide an embedded or secure document viewer.

## 22.2 PPT

Provide an appropriate presentation viewer or secure download option.

## 22.3 Notes

Render notes in a readable learning interface.

All resources must respect the same authorization rules.

# 23 Authentication and Routing

Use secure authentication.

Recommended authentication provider:

```
Supabase Auth
```

Role-aware routes:

```
/admin/*
/teacher/*
/student/*
```

Authentication flow:

```
Login
  |
  v
Authenticated User
  |
  v
Role Detection
  |
  +---- ADMIN ----> Admin Dashboard
  |
  +---- TEACHER --> Teacher Dashboard
  |
  +---- STUDENT --> Student Dashboard
```

Unauthorized users must be rejected.

# 24 Authorization Architecture

Authorization must be implemented on the backend.

## 24.1 Admin

```
Admin
  |
  +-- Full system access
```

## 24.2 Teacher

```
Teacher
  |
  v
Teacher Assignments
  |
  +-- Assigned Class
  |
  +-- Assigned Subject
  |
  +-- Optional Section
```

## 24.3 Student

```
Student
  |
  v
Student Enrollment
  |
  +-- Active Class
  |
  v
Class Subjects
  |
  v
Lessons
  |
  v
```

```
Published Materials
```

> Frontend route protection alone is not sufficient.
> Every sensitive backend endpoint must independently verify authentication and authorization.

## 25 API Architecture

Use versioned APIs.

```
/api/v1

/auth
    /login
    /logout
    /me
    /forgot-password

/admin
    /dashboard
    /users
    /academic-years
    /classes
    /sections
    /subjects
    /lessons
    /assignments
    /materials
    /announcements

/students
    /profile
    /classes
    /subjects
    /lessons
    /materials
    /progress

/teachers
    /profile
    /assignments
    /subjects
    /lessons
    /materials

/materials
    /upload
    /download
    /publish
    /delete
```

Use:

- Request validation.

- Response schemas.
- Authentication dependencies.
- Authorization dependencies.
- Pagination.
- Filtering.
- Sorting.
- Consistent error handling.

## 26 Frontend Architecture

Recommended technologies:

- React
- Vite
- TypeScript
- React Router
- TanStack Query
- Tailwind CSS

Recommended layouts:

```
AdminLayout
TeacherLayout
StudentLayout
```

Reusable components:

```
DataTable
SearchBar
FilterBar
Modal
Drawer
Form
FileUploader
VideoPlayer
PDFViewer
Sidebar
Header
StatCard
Pagination
ConfirmationDialog
EmptyState
LoadingState
ErrorState
```

# 27 Backend Architecture

Recommended backend stack:

- FastAPI
- Python
- Pydantic
- SQLAlchemy
- PostgreSQL

Suggested structure:

```
backend/
|
+-- app/
|   |
|   +-- api/
|   +-- models/
|   +-- schemas/
|   +-- services/
|   +-- repositories/
|   +-- auth/
|   +-- middleware/
|   +-- storage/
|   +-- utils/
|   +-- main.py
|
+-- migrations/
+-- tests/
+-- requirements.txt
```

Business logic should not be placed directly inside route handlers.

# 28 Complete Project Structure

```
academy-alys/
|
+-- frontend/
|   |
|   +-- src/
|       |
|       +-- components/
|       +-- layouts/
|       |   +-- AdminLayout/
|       |   +-- TeacherLayout/
|       |   +-- StudentLayout/
|       |
|       +-- pages/
|       |   |
|       |   +-- auth/
|       |       +-- AdminLogin/
|       |       +-- TeacherLogin/
|       |       +-- StudentLogin/
|       |
|       |   +-- admin/
|       |       +-- Dashboard/
|       |       +-- Students/
```

```
|       |       +-- Teachers/
|       |       +-- Classes/
|       |       +-- Subjects/
|       |       +-- Lessons/
|       |       +-- Materials/
|       |       +-- Timetable/
|       |       +-- Announcements/
|       |       +-- Reports/
|       |
|       |   +-- teacher/
|       |       +-- Dashboard/
|       |       +-- Subjects/
|       |       +-- Lessons/
|       |       +-- UploadMaterials/
|       |       +-- MyMaterials/
|       |
|       |   +-- student/
|       |       +-- Dashboard/
|       |       +-- Subjects/
|       |       +-- Lessons/
|       |       +-- LessonView/
|       |       +-- VideoPlayer/
|       |       +-- PDFViewer/
|       |       +-- Progress/
|       |
|       +-- services/
|       +-- hooks/
|       +-- contexts/
|       +-- types/
|       +-- utils/
|       +-- routes/
|
|   +-- package.json
|
+-- backend/
|   |
|   +-- app/
|       +-- api/
|       +-- models/
|       +-- schemas/
|       +-- services/
|       +-- repositories/
|       +-- auth/
|       +-- middleware/
|       +-- storage/
|       +-- utils/
|       +-- main.py
|
|   +-- migrations/
|   +-- tests/
|   +-- requirements.txt
|
+-- database/
|   +-- schema.sql
|   +-- seed.sql
|
+-- docs/
|   +-- architecture.md
|   +-- api.md
|   +-- database.md
|
```

```
+-- .env.example
+-- README.md
+-- docker-compose.yml
```

## 29 File Storage

Large learning files must not be stored directly in PostgreSQL.

Recommended:

```
Supabase Storage
        OR
S3-Compatible Object Storage
```

The database should store metadata such as:

- Bucket.
- File path.
- File name.
- File type.
- File size.
- Storage URL or reference.
- Uploaded by.
- Upload date.
- Publication status.

## 30 Security Requirements

The system must implement:

- Secure authentication.
- Role-based authorization.
- Object-level authorization.
- Input validation.
- File validation.
- File size restrictions.
- Secure file access.
- Audit logging.
- Secure password handling.
- Environment variables for secrets.
- API authorization.

Never:

- Store plaintext passwords.
- Expose authentication tokens.
- Trust frontend permissions.
- Allow arbitrary file uploads without validation.
- Commit secrets to GitHub.

# 31 Responsive Design

## 31.1 Desktop

- Persistent sidebar.
- Multi-column dashboards.
- Full data tables.
- Large content areas.

## 31.2 Tablet

- Collapsible sidebar.
- Responsive cards.
- Responsive tables.
- Stacked forms where necessary.

## 31.3 Mobile

- Navigation drawer.
- Stacked summary cards.
- Responsive forms.
- Mobile-friendly tables or record cards.
- Accessible controls.

# 32 UI/UX Principles

The platform should use a professional educational SaaS interface.

Prioritize:

- Clear visual hierarchy.
- Consistent spacing.
- Consistent component sizing.
- Accessible labels.
- Responsive layouts.
- Meaningful empty states.
- Loading states.

- Validation states.
- Confirmation dialogs.
- Clear navigation.

Every visible button must have a meaningful action.
Do not create generic dashboard components that are disconnected from actual workflows.

# 33 Important Data Rules

The following should never be hardcoded in the final implementation:

- Classes.
- Subjects.
- Teachers.
- Students.
- Lessons.
- Materials.
- Statistics.
- Teacher assignments.
- Student enrollments.

The UI may use seed/demo data during development.
The final application must retrieve data from the backend/database.

## 33.1 Historical Enrollment

Historical academic records must be preserved.
Example:

```
2025-2026 -> Class 9
2026-2027 -> Class 10
```

When a student moves from Class 9 to Class 10, do not overwrite the previous enrollment.
Create a new enrollment record for the new academic year.

# 34 Future AI Learning Layer

The AI layer should be implemented after the core LMS is stable.
The AI architecture can later be:

```
Teacher Material
       |
       v
Document Processing
       |
       v
Text Extraction
       |
```

```
       v
Chunking
       |
       v
Embeddings
       |
       v
Vector Database
       |
       v
RAG Retrieval
       |
       v
LLM
       |
       v
AI Learning Resources
```

Potential AI-generated resources:

- Simple explanations.
- Summaries.
- Short notes.
- Flashcards.
- Mind maps.
- Practice questions.
- AI tutor.

AI-generated content must remain associated with:

```
Class
  +
Subject
  +
Lesson
```

The AI layer must respect the same access-control system as teacher-uploaded content.

# 35 Development Roadmap

## 35.1 Phase 1 – Foundation

- Authentication.
- User roles.
- Database.
- Admin layout.
- Teacher layout.
- Student layout.
- Routing.

### 35.2 Phase 2 – Academic Structure

- Academic years.
- Classes.
- Sections.
- Subjects.
- Lessons.

### 35.3 Phase 3 – User Management

- Student creation.
- Student import.
- Student directory.
- Teacher creation.
- Teacher assignments.

### 35.4 Phase 4 – Teacher Portal

- Teacher dashboard.
- Assigned subjects.
- Lesson navigation.
- Video upload.
- PDF upload.
- PPT upload.
- Notes.
- Material management.

### 35.5 Phase 5 – Student Portal

- Student dashboard.
- Subject selection.
- Lesson selection.
- Lesson sidebar.
- Video player.
- PDF viewer.
- PPT viewer.
- Notes.
- Learning progress.

### 35.6 Phase 6 – Administration

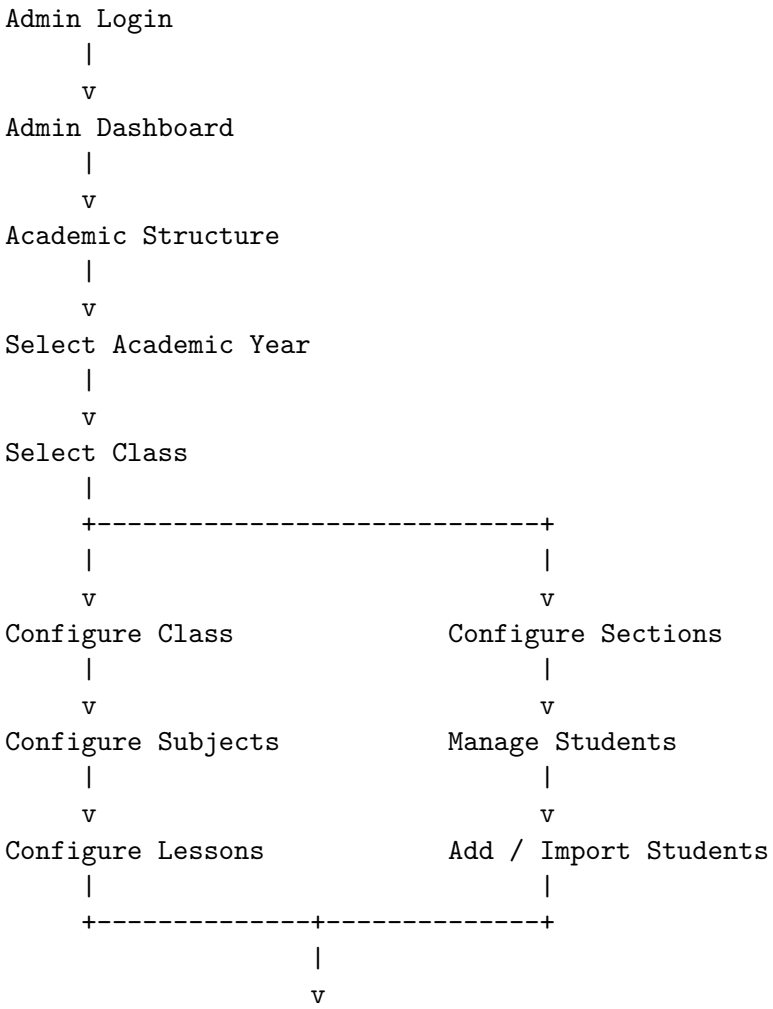
- Announcements.
- Timetable.
- Reports.
- Activity logs.

### 35.7 Phase 7 – AI

- RAG.
- AI summaries.
- Short notes.
- Flashcards.
- Mind maps.
- Practice questions.
- AI tutor.

## 36 Complete End-to-End System Flow

### 36.1 Admin Flow



```
Manage Teachers
       |
       v
Assign Class + Subject
       |
       v
System Ready
```

### 36.2 Teacher Flow

```
Teacher Login
      |
      v
Teacher Dashboard
      |
      v
Assigned Class
      |
      v
Assigned Subject
      |
      v
Lesson
      |
      v
Upload Material
      |
      v
Preview
      |
      v
Publish
```

### 36.3 Student Flow

```
Student Login
      |
      v
Student Dashboard
      |
      v
Assigned Class
      |
      v
Available Subjects
      |
      v
Subject
      |
      v
Lessons
      |
      v
Lesson
      |
```

```
      v
Home / Summary
      |
      +-------------------+
      |         |         |
      v         v         v
   Videos     PDFs      Notes
      |
      v
Video Player
      |
      v
Learn
```

## 37 Permission Flow

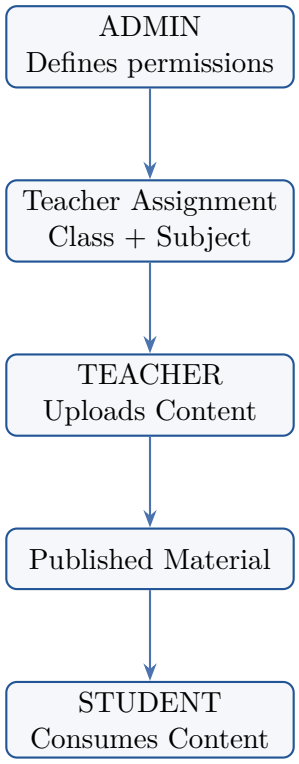



Figure 2: Core permission and content flow

## 38 Final Acceptance Criteria

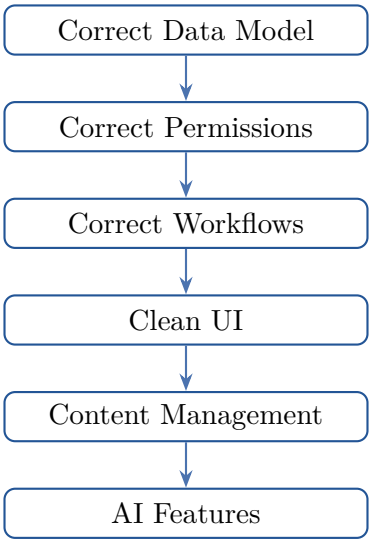
The project is considered functionally complete when all of the following are satisfied:

1. Three separate login portals work.
2. Admin can create and manage students.
3. Admin can import students in bulk.
4. Admin can create teachers.
5. Admin can assign teachers to specific classes and subjects.
6. Students are linked to academic classes.

7. Classes can contain sections.
8. Subjects can be configured per class.
9. Lessons belong to subjects and classes.
10. Teachers can upload materials only to authorized class-subject combinations.
11. Students can see only their assigned class.
12. Students can see only subjects available to their class.
13. Students can see lessons belonging to those subjects.
14. Students can see only published materials they are authorized to access.
15. Video playback works.
16. PDF viewing works.
17. Material counts in the lesson sidebar are dynamic.
18. Class selection is preserved throughout Admin class-specific workflows.
19. Authentication is enforced.
20. Backend authorization is enforced.
21. Sensitive credentials are never exposed.
22. Historical academic enrollment data is preserved.
23. The application is responsive.
24. The codebase is modular and maintainable.
25. Database migrations are provided.
26. Seed/demo data is provided.
27. API documentation is provided.
28. Environment configuration is documented.
29. Frontend and backend can be deployed independently.

## 39 Master Development Principle

The development priority should be:



The platform must remain a functional LMS even if the AI layer is completely disabled. The AI layer is an enhancement, not a dependency for the core platform.

## 40 Master Handover Prompt

The following prompt can be given directly to a developer or coding agent.

**PROJECT: ACADEMYALYS**
Build AcademyAlys, a production-ready role-based educational learning management platform for Classes 6–10.
The platform has three completely separate portals:

1. ADMIN
2. TEACHER
3. STUDENT

The complete system follows:

$$\text{ADMIN} \rightarrow \text{PERMISSIONS} \rightarrow \text{TEACHER} \rightarrow \text{CONTENT} \rightarrow \text{STUDENT}$$

Admin defines the academic structure and permissions.
Teachers can manage content only for classes and subjects explicitly assigned to them.
Students can access only the class, subjects, lessons and published materials available to their academic enrollment.
Never rely only on frontend visibility for authorization. All permissions must be enforced on the backend and database layer.
The academic hierarchy is:

$$\text{Academic Year} \rightarrow \text{Class} \rightarrow \text{Section} \rightarrow \text{Subject} \rightarrow \text{Unit} \rightarrow \text{Lesson} \rightarrow \text{Topic} \rightarrow \text{Material}$$

Implement:

- Secure authentication.
- Role-based routing.
- Admin Portal.
- Teacher Portal.
- Student Portal.
- Academic year management.
- Class management.
- Section management.
- Subject management.
- Lesson management.
- Student management.
- Bulk student import.
- Teacher management.
- Teacher class-subject assignments.
- Video uploads.
- PDF uploads.
- PPT uploads.
- Notes.
- Student lesson interface.
- Video player.

**AcademyAlys**

Educational Learning Management Platform

Complete Architecture and Project Handover Specification